WebAssembly 勉強会 最終発表

WASM は本当に速いのか？

2つのプロダクトで検証してわかったこと

あべ

目次

1. 何を作ったのか
2. なぜ作ったのか
3. 遭遇した課題
4. 課題をどう解決したか
5. WASM / AssemblyScript ナレッジ共有

1. 何を作ったのか

作ったもの: 2つのプロダクト

image-filter

- 画像にグレースケールフィルターをかけるアプリ
- **ピクセル処理**（数値演算が主体）
- React + AssemblyScript
- WASM vs JS のベンチマーク比較

diff-tool

- テキストの差分を比較するツール
- **文字列処理**（比較・分岐が主体）
- React + AssemblyScript
- Myers diff アルゴリズムを WASM と JS で実装

技術スタック

	image-filter	diff-tool
フレームワーク	React 19 + Vite	React 19 + Vite 8
WASM 言語	AssemblyScript	AssemblyScript
アルゴリズム	ピクセルごとのグレースケール変換	Myers diff（最短編集距離）
データ型	`Uint8Array`（数値）	`string`（文字列）

2. なぜ作ったのか

動機

- 「WASM = 速い」は本当か？
  - 調べると速いという記事が多いが、実際に検証した人は少ない
- **データの種類**によって結果が変わるのでは？
  - 数値演算が得意なのはわかる
  - 文字列処理はどうなのか？
- 2つの異なる特性のプロダクトを作って比較することで、
  WASM の得意・不得意を実測で明らかにしたかった

3. 遭遇した課題

課題 1: Vite 8 と AssemblyScript の互換性

AssemblyScript が生成する `release.js`（ラッパー）を Vite 8 がパースできない

```
[PARSE_ERROR] Error: Duplicated export 'diff'
[PARSE_ERROR] Error: Duplicated export 'memory'
```

原因: Vite 8 がトランスパイラを esbuild → oxc に切替え、
oxc のパーサーが AssemblyScript のエクスポートパターンを処理できなかった

課題 2: WASM と JS 間のデータ受け渡し

WASM は自分の線形メモリ（`ArrayBuffer`）しか見えない

```
JS string "hello"
    ↓ lowerString（1文字ずつメモリに書き込み）
WASM Memory [0x68, 0x00, 0x65, 0x00, ...] ← UTF-16
    ↓ diff計算
WASM Memory 上の結果 JSON 文字列
    ↓ liftString（1文字ずつ読み出し）
JS string → JSON.parse → JS object
```

image-filter ではピクセルデータが最初から `ArrayBuffer` なのでコピー不要だった

課題 3: WASM が JS より遅かった（diff-tool）

ベンチマーク結果:

行数	WASM	JS	倍率
100	3.10 ms	0.40 ms	JS の 7.7x 遅い
500	9.80 ms	2.00 ms	JS の 4.9x 遅い
1,000	22.60 ms	4.10 ms	JS の 5.5x 遅い
5,000	407.30 ms	122.70 ms	JS の 3.3x 遅い
10,000	1,592 ms	411.70 ms	JS の 3.9x 遅い

全行数で一貫して WASM が遅い

4. 課題をどう解決したか

Vite 8 互換性の解決

解決策: AssemblyScript のバインディングモードを変更

```
// asconfig.json
{
  "options": {
    "bindings": "raw"  // "esm" → "raw" に変更
  }
}
```

- `"esm"`: 自動インスタンス化 + 重複エクスポートパターン → oxc でエラー
- `"raw"`: `instantiate()` 関数だけをエクスポート → 問題なし

加えて `vite-plugin-wasm` を導入し、WASM ファイルの読み込みを Vite に任せた

ボトルネックの特定

`release.js` にログを仕込んで各フェーズの時間を計測

```
[WASM] lower: 0.80ms | exec: 1670.20ms | lift: 25.70ms
```

- **lower（JS→WASM変換）**：0.80ms ← ほぼゼロ
- **exec（diff計算本体）**：1670.20ms ← **ここが98.4%**
- **lift（WASM→JS変換）**：25.70ms ← 小さい

予想に反して、文字列コピーではなく **WASM 内の計算自体が遅かった**

5. ナレッジ共有

なぜ image-filter では WASM が速く、diff-tool では遅いのか

image-filter（WASM が速い）

- データが `Uint8Array`（数値の配列）
- メモリを共有して直接アクセス
- 全ピクセルに同じ演算を繰り返す
- 分岐がほぼない

diff-tool（WASM が遅い）

- データが `string`（文字列）
- 配列の動的確保・コピーが多い
- 分岐パターンが入力データに依存
- AssemblyScript のランタイムオーバーヘッド

V8 JIT vs WASM AOT

	V8 (JavaScript)	WASM
コンパイル	実行時に JIT コンパイル	事前にコンパイル済み (AOT)
最適化	実行中にコードの挙動を観察して適応的に最適化	コンパイル時に決定、実行時の最適化なし
得意	分岐の多いアルゴリズム、文字列操作	単純な数値演算の大量繰り返し
配列・文字列	何十年もかけて最適化された組み込み実装	AssemblyScript の自前ランタイム

WASM が常に速いわけではない

WASM が向いている / 向いていないケース

向いている

- 画像・音声・動画の処理（ピクセル、サンプルデータ）
- 暗号化・圧縮（CPU バウンドな数値演算）
- 物理シミュレーション、ゲームロジック
- C/C++/Rust の既存ライブラリをブラウザで使いたい場合

向いていない

- 文字列処理が主体のアルゴリズム
- DOM 操作（そもそも WASM から直接触れない）
- I/O バウンドな処理（API 呼び出し等）

JS ⇔ WASM のデータ受け渡しまとめ

データ型	受け渡し方法	コスト
数値（`i32`, `f64`）	関数の引数/戻り値でそのまま	ほぼゼロ
バイト配列（`Uint8Array`）	`WebAssembly.Memory` を共有	コピー不要
文字列（`string`）	1文字ずつメモリにコピー（UTF-16）	入出力のたびにコピー発生
オブジェクト	JSON シリアライズ経由	コピー + パースのコスト

データが最初から `ArrayBuffer` 上にあるなら WASM が有利

まとめ

- WASM は**万能ではない** — データの種類と処理の特性で有利・不利がある
- 数値演算・バイト配列操作 → WASM が有利（image-filter で実証）
- 文字列処理・複雑な分岐 → JS (V8 JIT) が有利（diff-tool で実証）
- ボトルネックは推測ではなく**計測**で特定すべき
  - 文字列コピーが遅いと思ったら、実際は WASM 内の計算自体が遅かった
- AssemblyScript は手軽だが、パフォーマンスを求めるなら Rust/C++ も検討

ありがとうございました

リポジトリ: Wasm-Project/

- `image-filter/` — 画像フィルター（WASM が速い例）
- `diff-tool/` — テキスト差分比較（JS が速い例）